

Grupos peligrosos (docker, lxd, disk)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Algunos grupos conceden capacidades que equivalen a root aunque tu usuario no lo sea. docker y lxd permiten arrancar un contenedor que monta el sistema de ficheros del host y escribir en el como root. disk da acceso crudo al dispositivo (leer /etc/shadow con debugfs). adm permite leer todos los logs. Es el atajo mas rapido del privesc-modelo.

Estos grupos existen para administrar subsistemas, pero su poder se traduce trivialmente en root: si controlas el demonio de contenedores puedes montar / del host; si lees el disco crudo, saltas todos los permisos de fichero. Por eso id/groups es lo primero que se mira.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Nada mas orientarte con id/groups (privesc-modelo). Si sales en docker/lxd/disk/adm, prioriza este vector sobre todo lo demas.

Cómo identificarla

- id muestra tu usuario en docker, lxd/lxc, disk o adm.
- El binario docker/lxc responde (el demonio esta corriendo).
- Para disk: debugfs puede abrir el dispositivo raiz.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

id para ver tus grupos; si sale docker, prueba docker run -v /:/mnt --rm -it alpine chroot /mnt sh.

Evidencia esperada: Una shell dentro del chroot con acceso de escritura a / del host real.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
docker run -v /:/mnt --rm -it alpine chroot /mnt sh
```

Objetivo y contexto: Si estas en el grupo docker, arrancas un contenedor que monta TODO el disco del host en /mnt y haces chroot: ya eres root sobre los ficheros reales del host.

Resultado esperado: Una shell root dentro del chroot con acceso a / del host. Desde ahí lees flags, editas /etc/passwd o pones SUID a bash en el host real.

Capa 2: prueba documentada

```
debugfs -w /dev/sda1
```

Objetivo y contexto: Con el grupo disk accedes al dispositivo en crudo, saltando permisos de fichero. debugfs -w permite leer y escribir el sistema de ficheros directamente.

Resultado esperado: Un prompt debugfs. 'cat /etc/shadow' desde ahí ignora permisos. Ajusta /dev/sdaX al dispositivo raíz real (mira lsblk/mount).

Anatomía de los payloads

La primera prueba es `docker run -v /:/mnt --rm -it alpine chroot /mnt sh`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Una shell dentro del chroot con acceso de escritura a / del host real.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Mira tus grupos con id.
- docker: arranca un contenedor montando / del host y chroot para escribir como root.
- lxd: importa una imagen y montala con security.privileged para acceder al host.
- disk: usa debugfs sobre el dispositivo raíz para leer /etc/shadow o escribir.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- id no muestra ninguno de esos grupos.

- El demonio correspondiente (docker/lxd) no esta corriendo pese a estar en el grupo.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Pertenecer a ciertos grupos es root casi directo: docker/lxd montan el disco del host; disk lo lee crudo. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(group-abuse-linux\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.